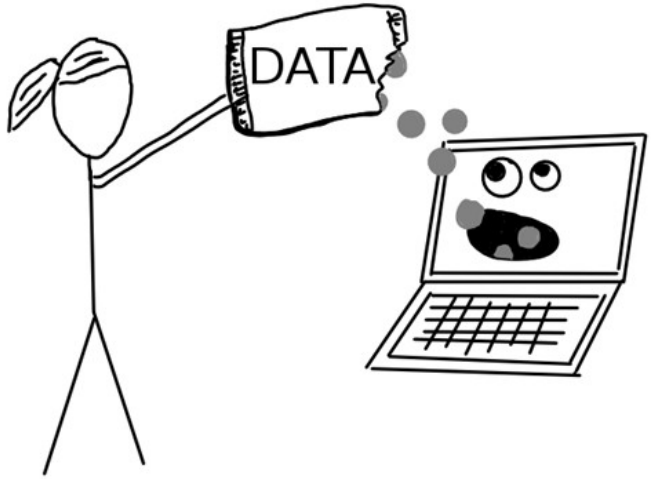
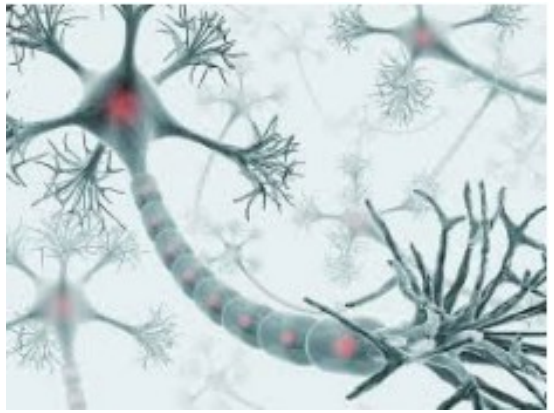
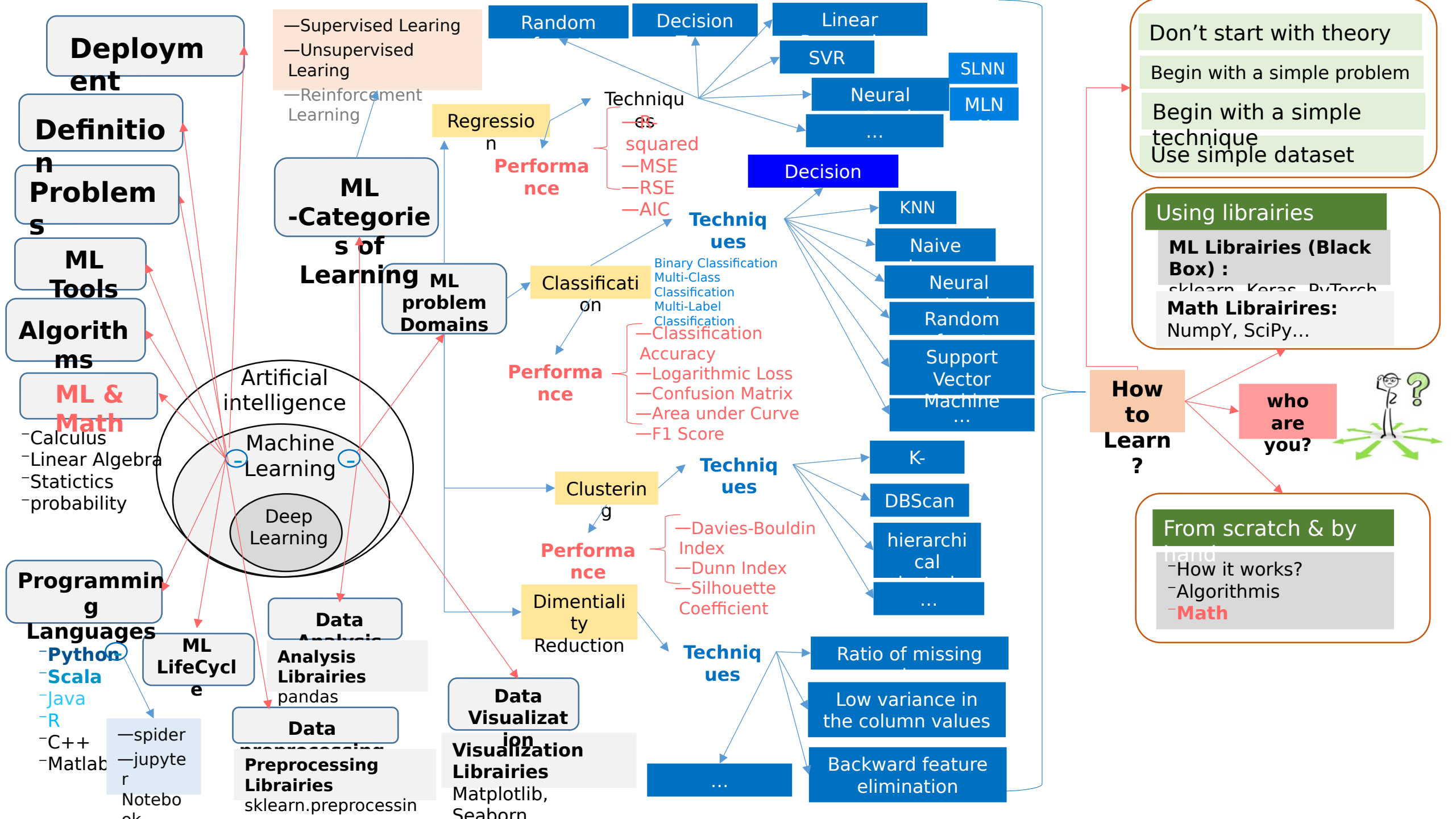


# Neural Network

## Multilayer Neural Network

**Abdelwahab Naji - Enseignant chercheur**
Abdelwahab.naji@gmail.com
Laboratoire Signaux, Systèmes Distribués et Intelligence Artificielle (SSDIA)
ENSET de Mohammedia, Université Hassan II de Casablanca
https://www.youtube.com/c/AbdelwahabNaji


Deployment
Definition
Problems
ML Tools
Algorithms
ML & Math
Calculus
Linear Algebra
Statictics
probability
Artificial intelligence
Machine Learning
Deep Learning
Programming Languages
Python
Scala
Java
R
C++
Matlab
ML LifeCycle
spider
jupyter Notebook
Data Analysis
Analysis Librairies
pandas
Data preprocessing
Preprocessing Librairies
sklearn.preprocessin
Data Visualization
Visualization Librairies
Matplotlib, Seaborn
ML -Categories of Learning
Supervised Learing
Unsupervised Learing
Reinforcement Learning
ML problem Domains
Regression
Performance
Techniques
R squared
MSE
RSE
AIC
Random
Decision
Linear
SVR
Neural
SLNN
MLN
...
Classification
Binary Classification
Multi-Class Classification
Multi-Label Classification
Performance
Classification Accuracy
Logarithmic Loss
Confusion Matrix
Area under Curve
F1 Score
Techniques
Decision
KNN
Naive
Neural
Random
Support Vector Machine
...
Clustering
Performance
Davies-Bouldin Index
Dunn Index
Silhouette Coefficient
Techniques
K-
DBScan
hierarchical
...
Dimentiality Reduction
Techniques
Ratio of missing
Low variance in the column values
Backward feature elimination
...
How to Learn ?
Don't start with theory
Begin with a simple problem
Begin with a simple technique
Use simple dataset
Using librairies
ML Librairies (Black Box) :
sklearn, Keras, PyTorch
Math Librairires:
NumpY, SciPy...
who are you?
From scratch & by hand
How it works?
Algorithmis
Math

# Mise en œuvre de Neural Network: les étapes à suivre

- NN (un seul neurone) avec tensorflow, keras
  - Problème linéairement séparable vs problème non linéairement séparable
  - Loss function
  - Exemple 1: generated data pour un problème linéairement séparable
  - Exemple 2: diabetes dataset
- NN (un seule Neurone) from scracth avec gradient descent
  - Traiter les mêmes exemples
- NN: Multi – Layer (plusieurs neurones) avec tensorflow, keras
  - Problème non linéairement séparable
    - Exemple 1: XOR problem
    - **Exemple 2:  fashion classification problem**
    - Exemple 3: hand written digits classification problem
    - Exemple 4: diabetes classification problem
- NN: Multi – Layer (plusieurs neurones) from scratch avec gradient descent
  - Traiter les mêmes exemples

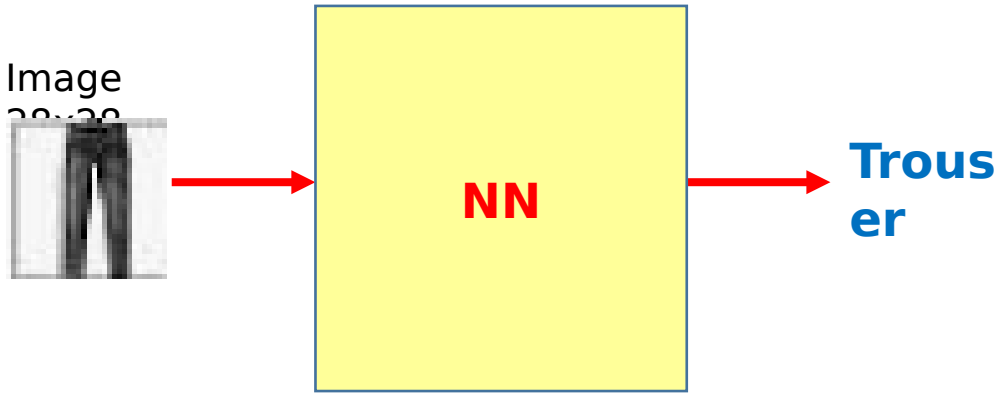
# Dataset

| Class | Description |
|---|---|
| 0 | T-shirt/top |
| 1 | Trouser |
| 2 | Pullover |
| 3 | Dress |
| 4 | Coat |
| 5 | Sandal |
| 6 | Shirt |
| 7 | Sneaker |
| 8 | Bag |
| 9 | Ankle boot |

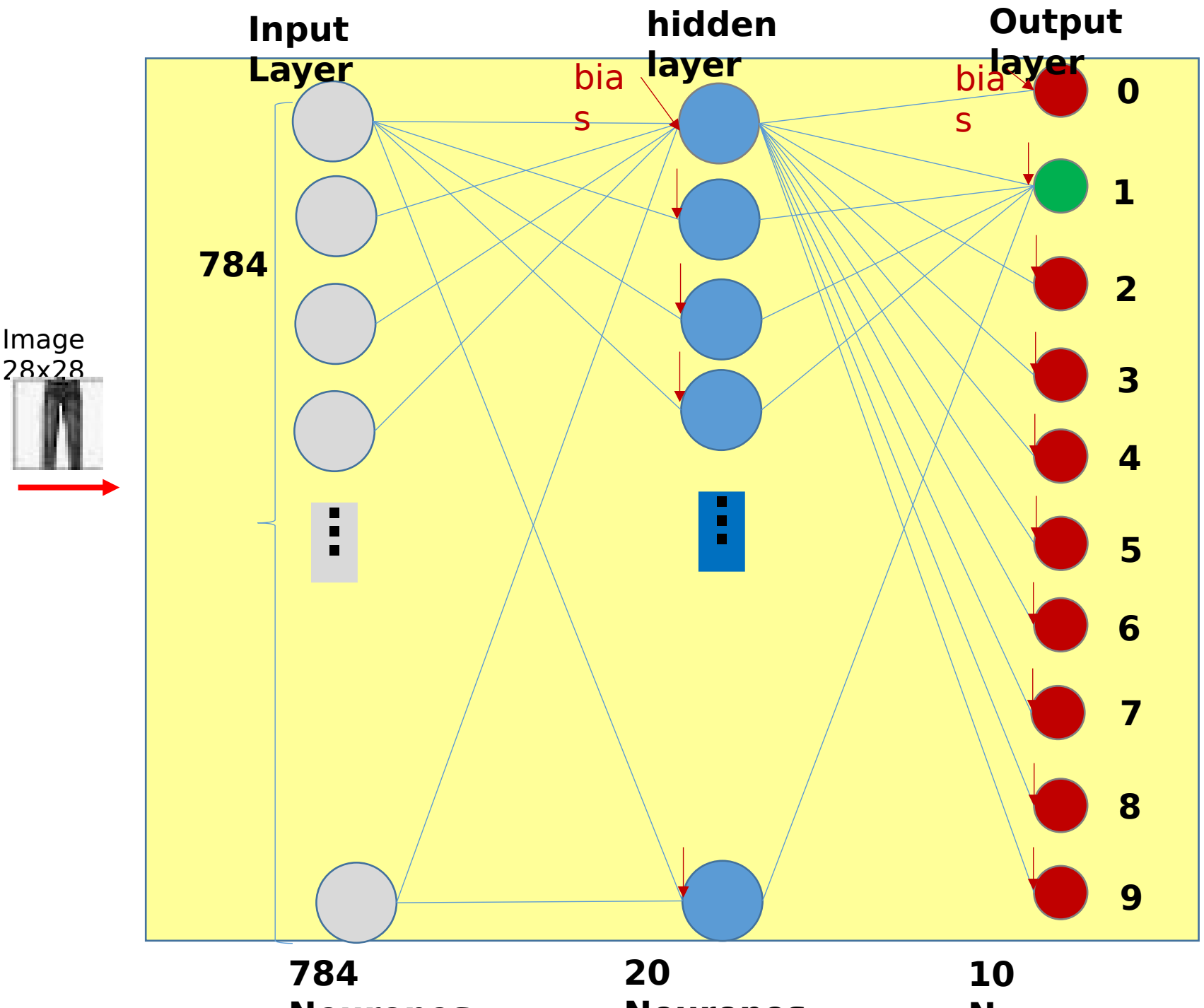
https://keras.io/api/datasets/fashion_mnist/

# Problem

Human

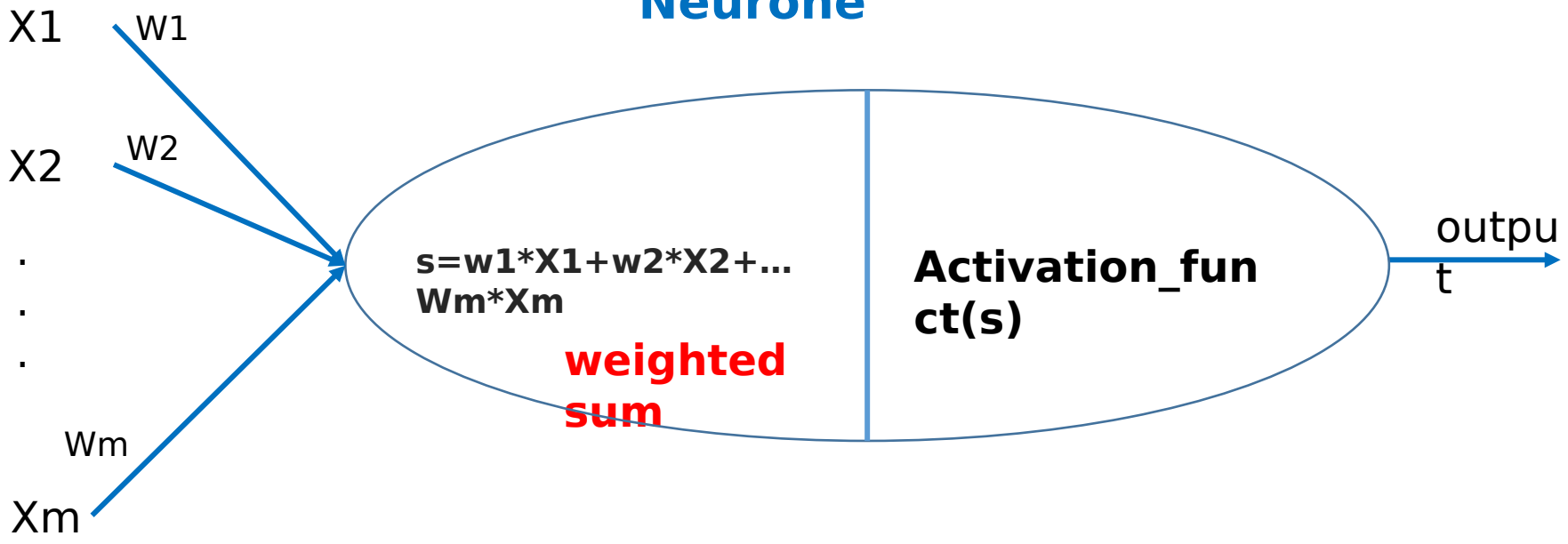



Computer




| Class | Description |
|---|---|
| 0 | T-shirt/top |
| 1 | Trouser |
| 2 | Pullover |
| 3 | Dress |
| 4 | Coat |
| 5 | Sandal |
| 6 | Shirt |
| 7 | Sneaker |
| 8 | Bag |
| 9 | Ankle boot |

# Neurone

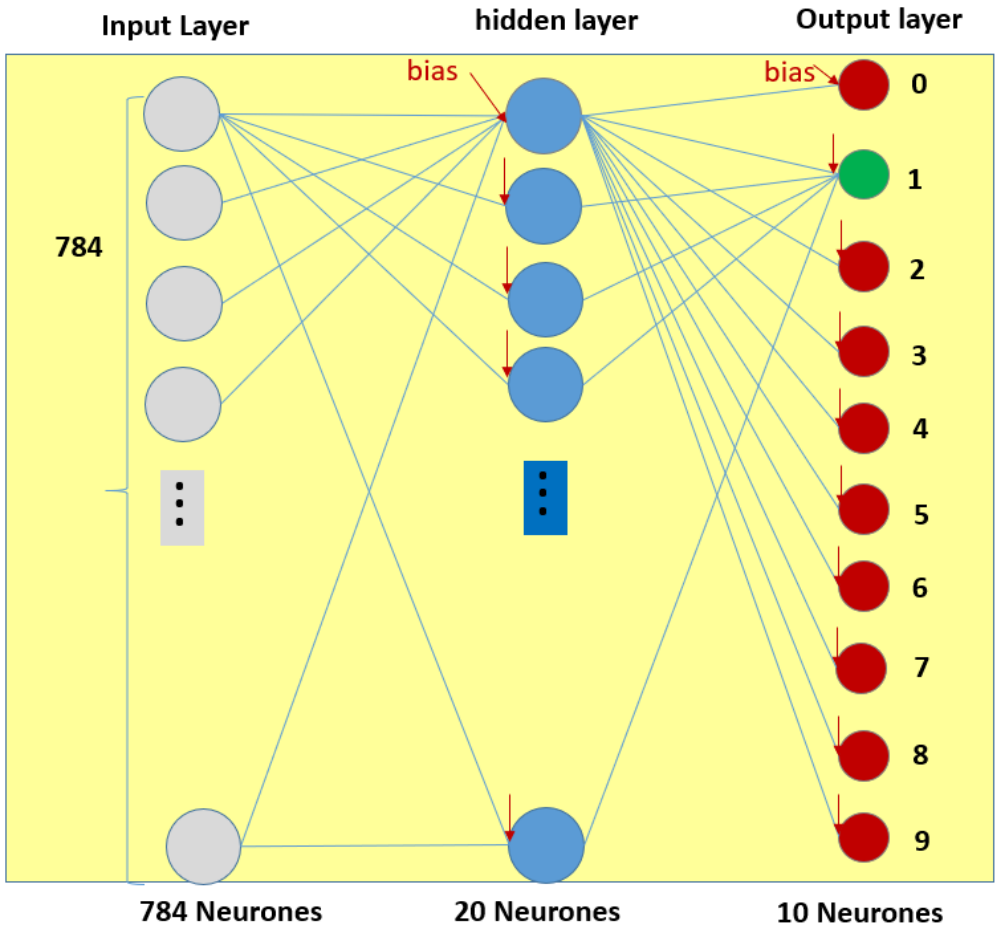



**Traitement**

1. Le neurone reçoit des entrées
2. Calcule la somme pondérée $\sum Wi*Xi$
3. Calcule la sortie à l'aide d'une fonction d'activation

**Element**

1. Inputs
2. Weights
3. Activation function
   - Sigmoid
   - Relu
   - Softmax




**Learning**

1. fit(dataset): update weights

**Prediction**

2. predict(newdata): output

**Element**

1. Input Layer
2. **Hidden Layers**
3. Output Layer

# Multilayer Neural Network by using tensorflow, keras

# Les étapes à suivre pour créer le modèle

1. **Data**
   - Get data, Features (X), target (y)
   - Visualize data
   - Split Data (training data, test data)
2. **Model**
   - Create the model, compile the model: keras, tensorflow
     - Optimizer function: adam
     - Loss function: sparse_categorical_crossentropy
   - Train the model
   - Parameters
3. **Prediction**
4. **Evaluation**

# 1. Dataset

```
import keras
```

```
dataset = keras.datasets.fashion_mnist
(X_train, y_train), (X_test, y_test) =
dataset.load_data()
```

```
X_train.shape
```

**(60000, 28, 28)**

```
y_train.shape
```

**(60000, )**

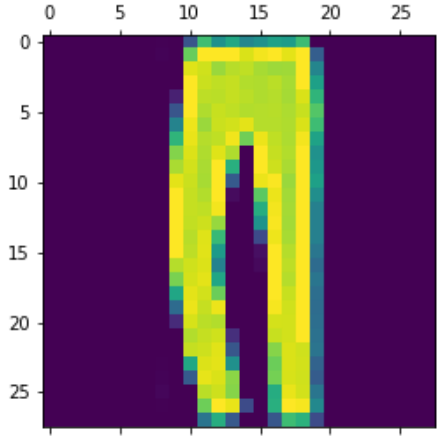
```
X_test[2]
```

```
array([[  0,   0,   0,   0,   0,   0,   0,   0,   1,   0,  67, 177, 129,
        153, 117, 129, 146, 141, 175,   0,   0,   0,   0,   0,   0,   0,
          0,   0],
       [  0,   0,   0,   0,   0,   0,   0,   0,   2,   0, 196, 255, 255,
        255, 241, 251, 255, 245, 255,  76,   0,   0,   0,   0,   0,   0,
          0,   0],
       [  0,   0,   0,   0,   0,   0,   0,   0,   0,   0, 243, 237, 217,
        228, 231, 222, 218, 219, 255, 116,   0,   0,   0,   0,   0,   0,
          0,   0],
       [  0,   0,   0,   0,   0,   0,   0,   0,   0,   0, 255, 235, 231,
        233, 226, 225, 229, 222, 255, 145,   0,   0,   0,   0,   0,   0,
          0,   0],
       [  0,   0,   0,   0,   0,   0,   0,   0,   0,  23, 255, 230, 229,
        233, 229, 225, 227, 221, 241, 167,   0,   0,   0,   0,   0,   0,
          0,   0],
       [  0,   0,   0,   0,   0,   0,   0,   0,   0,  67, 255, 225, 234,
        231, 226, 227, 228, 224, 234, 187,   0,   0,   0,   0,   0,   0,
          0,   0],
       [  0,   0,   0,   0,   0,   0,   0,   0,   0, 114, 249, 217, 235,
        238, 245, 227, 227, 225, 241, 176,   0,   0,   0,   0,   0,   0,
          0,   0],
       [  0,   0,   0,   0,   0,   0,   0,   0,   0, 167, 247, 227, 231,
        250, 200, 248, 231, 224, 251, 147,   0,   0,   0,   0,   0,   0,
          0,   0],
       [  0,   0,   0,   0,   0,   0,   0,   0,   0, 208, 242, 225, 243,
        225,   0, 255, 233, 226, 255, 129,   0,   0,   0,   0,   0,   0,
          0,   0]
```

```
y_test[2]
```

**1**

```
plt.matshow(X_test[2])
```




```
X_train = X_train/255
X_test = X_test/255
```

| Class | Description |
|---|---|
| 0 | T-shirt/top |
| 1 | Trouser |
| 2 | Pullover |
| 3 | Dress |
| 4 | Coat |
| 5 | Sandal |
| 6 | Shirt |
| 7 | Sneaker |
| 8 | Bag |
| 9 | Ankle boot |

# Les étapes à suivre pour créer le modèle

1. **Data**
   - Get data, Features (X), target (y)
   - Visualize data
   - Split Data (training data, test data)
2. **Model**
   - Create the model, compile the model: keras, tensorflow
     - Optimizer function: adam
     - Loss function: sparse_categorical_crossentropy
   - Train the model
   - Parameters
3. **Prediction**
4. **Evaluation**

**Plusieurs fonctions pour évaluer cost**

1. Binary
2. BinaryCrossentropy class
3. CategoricalCrossentropy class
4. SparseCategoricalCrossentropy class
5. Poisson class
6. binary_crossentropy function
7. categorical_crossentropy function
8. **sparse_categorical_crossentropy function**
9. ...

Optimzer functions sont basés sur l’algorithme Grandient Descent

Plusieurs variantes de l’algorithme sont disponibles

1. Batch Gradient Descent
2. Stochastic Gradient Descent
3. Mini Batch Gradient Descent
4. Adagrad
5. Adamax
6. RMSprop
7. Adam
8. ...

# 2. Model

```
from keras.models import Sequential
from keras.layers import Flatten, Dense, Activation
```

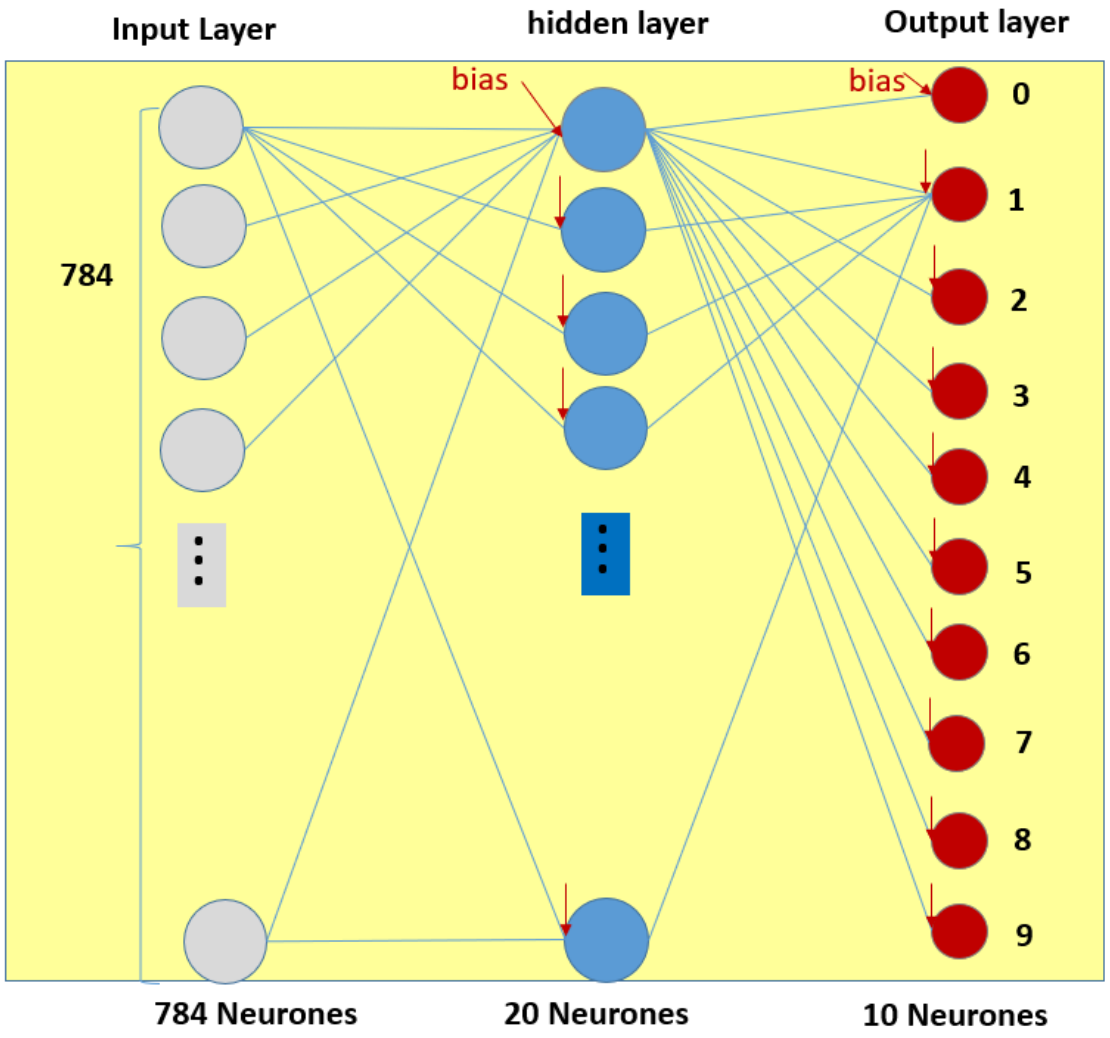
```
model = Sequential()
```

```
input_layer=Flatten(input_shape=[28, 28])
model.add(input_layer)
hidden_layer=Dense(20,activation="relu")
model.add(hidden_layer)
output_layer=Dense(10, activation="softmax")
model.add(output_layer)
print(model.summary())
print(model.get_weights()) #initial weights
```

```
model.compile(loss="sparse_categorical_crossentropy",
              optimizer="adam",
              metrics=["accuracy"])
```

```
model.fit(X_train, y_train,epochs=10)
```

```
print(model.summary())
print(model.get_weights()) #learned weights
```



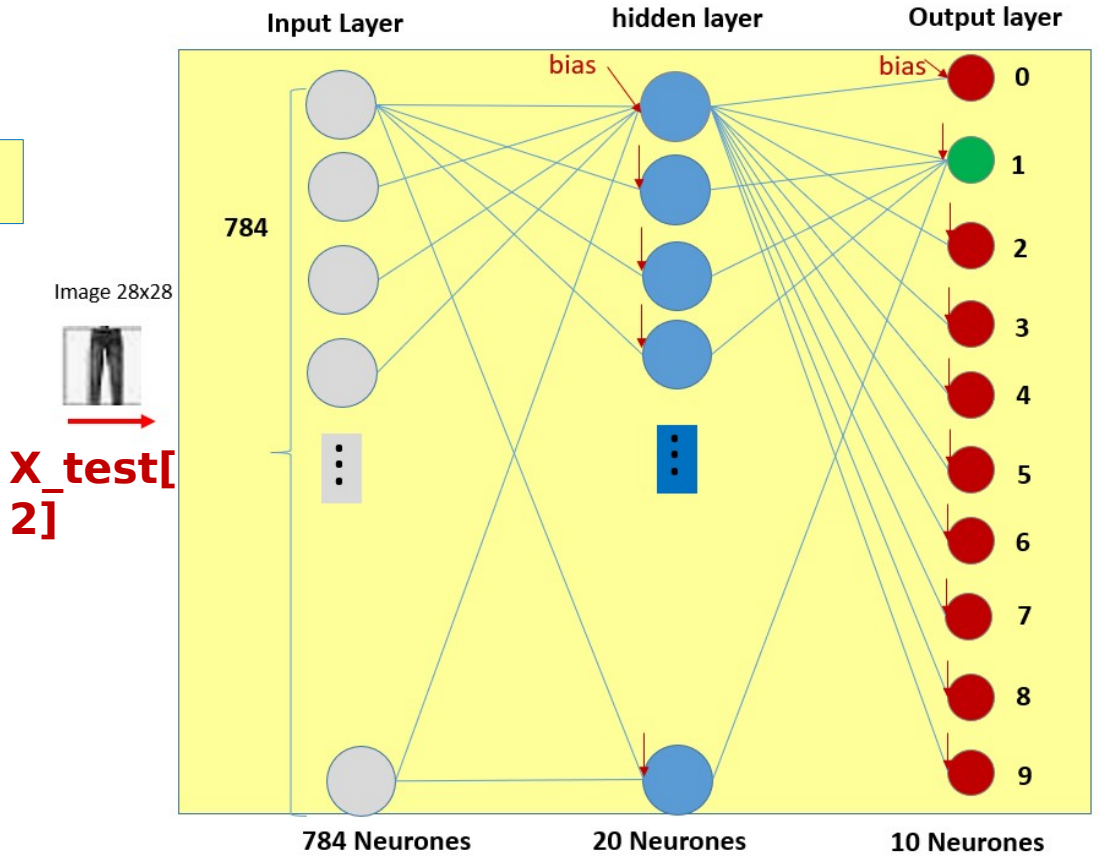
# Les étapes à suivre pour créer le modèle

1. **Data**
   - Get data, Features (X), target (y)
   - Visualize data
   - Split Data (training data, test data)
2. **Model**
   - Create the model, compile the model: keras, tensorflow
     - Optimizer function: adam
     - Loss function: sparse_categorical_crossentropy
   - Train the model
   - Parameters
3. **Prediction**
4. **Evaluation**

# 3. Prediction

```
yp=model.predict(np.array([X_test[2]]))
```

```
array([[0., 1., 0., 0., 0., 0., 0., 0.,
0., 0.]]
```




```
classes =
["T-shirt/top","Trouser","Pullover","Dress","Coat","Sandal","Shirt","Sneaker","Bag","Ankle
```

```
classes[np.argmax(yp)]
```

```
yp = model.predict(X_test)
```

```
classes[np.argmax(yp[2])]
```

# Les étapes à suivre pour créer le modèle

1. **Data**
   - Get data, Features (X), target (y)
   - Visualize data
   - Split Data (training data, test data)
2. **Model**
   - Create the model, compile the model: keras, tensorflow
     - Loss function: sparse_categorical_crossentropy
   - Train the model
   - Parameters
3. **Prediction**
4. **Evaluation**

# 2. Evaluation

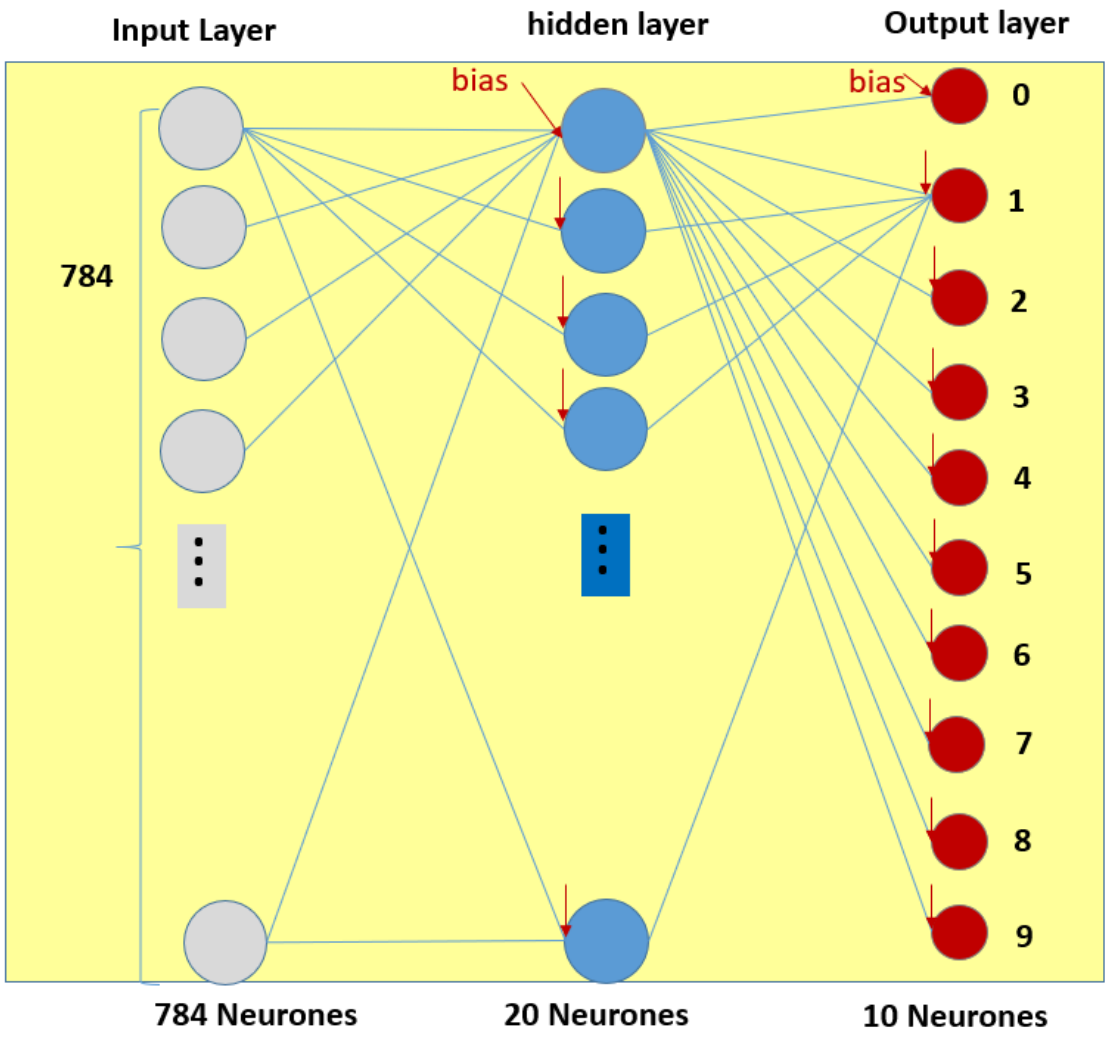
```
print(model.metrics_names)
```

['loss',
'accuracy']

```
metrics=model.evaluate(X_test,y_test)

print(metrics)
```

[0.4006, 0.8568]



# Exercice 1

1. Refaire le même travail
2. Essayes d’améliorer la performance du modèle
3. Ajouter une autre couche cachée
4. Evaluer le modèle obtenu

# Exercice 2

1. **Traiter le problème de classification des digits**

# Exercice 3

1. **Traiter le problème de classification des diabetes**